



Society of Petroleum Engineers

SPE-229512-MS

Agentic Retrieval Augmented Generation for Drilling: A Real-Time Framework for Dynamic Insights and Visualization

Hakim Arabi, Minyue Liu, Ganesh Arunachalam, and Lina Xu, SLB, Houston, Texas, USA

Copyright 2025, Society of Petroleum Engineers DOI [10.2118/229512-MS](https://doi.org/10.2118/229512-MS)

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

This paper was selected for presentation by an SPE program committee following review of information contained in an abstract submitted by the author(s). Contents of the paper have not been reviewed by the Society of Petroleum Engineers and are subject to correction by the author(s). The material does not necessarily reflect any position of the Society of Petroleum Engineers, its officers, or members. Electronic reproduction, distribution, or storage of any part of this paper without the written consent of the Society of Petroleum Engineers is prohibited. Permission to reproduce in print is restricted to an abstract of not more than 300 words; illustrations may not be copied. The abstract must contain conspicuous acknowledgment of SPE copyright.

Abstract

Large Language Models (LLMs) hold immense potential for advanced data retrieval and insight generation, yet integrating these capabilities into real-time industrial analytics poses a significant challenge. Traditional business intelligence (BI) systems relying on static dashboards and complex data transformations, often struggle to meet the dynamic demands of modern drilling operations. This paper introduces an Agentic Retrieval-Augmented Generation (RAG) framework that unifies structured data via knowledge graphs and domain-specific reasoning into a single conversational interface. By orchestrating multi-step queries and generating interactive visualizations, the framework delivers timely, context-aware insights, reducing user overhead and enhancing decision-making. Demonstrations in drilling workflows highlight marked improvements in safety, efficiency, and scalability, underscoring the transformative potential of LLM-driven RAG systems for next-generation industrial analytics.

Introduction

The oil and gas industry is undergoing a digital transformation, with real-time insights becoming crucial for operational efficiency, safety, and cost management. Traditional business intelligence (BI) systems have served as the backbone of industrial analytics but are increasingly inadequate for handling the scale, complexity, and dynamic requirements of modern drilling operations. These systems rely heavily on complex data pipelines involving multiple database layers and processing platforms that aggregate and process data. While effective for static reporting, such systems often lead to inefficiencies and delayed insights in rapidly changing operational contexts.

Retrieval-Augmented Generation (RAG) (Lewis et al. 2020) systems powered by Large Language Models (LLM) (Minaee et al. 2024; Singh 2023; Zhao et al. 2024) provide a paradigm shift by integrating real-time data retrieval with natural language interfaces. However, traditional RAG systems often lack adaptability and scalability, limiting their effectiveness in handling the diverse, interconnected datasets found in drilling operations.

The Agentic RAG framework addresses these limitations by introducing a multi-agent orchestration design and leveraging a fine-tuned knowledge graph. This knowledge graph acts as a unified schema model, consolidating data from diverse sources and providing a global view of the data ecosystem. By integrating

this with the RAG framework, the system delivers real-time, context-aware insights tailored to the specific needs of all stakeholders in rig operations. These include company men, drillers, remote operation engineers, and decision-makers. By simplifying data access and enhancing contextual understanding, the framework fosters more informed and efficient decision-making across the domain.

In this paper, we begin by providing an overview of the drilling data ecosystem and outlining the key challenges associated with it. We then demonstrate how the Agentic RAG framework effectively addresses these challenges. Following this, we present the graph ontology schema and describe our tailored Agentic RAG design that leverages this ontology. Finally, we showcase early results and validate the system within a specific application, highlighting its impact on operational efficiency and decision-making in drilling operations.

Background

Complexity of Data Ecosystem and BI Limitations

Drilling operations involve complex workflows that rely on real-time data acquisition, aggregation, and interpretation to ensure safe and efficient execution. Data generated during drilling is collected from various sources, such as planning contextual data, Bottom Hole Assembly (BHA) tools including Measurements While Drilling (MWD) tools, Logging While Drilling (LWD) tools, Rotary Steerable System (RSS) tools, surface sensors, and trajectory monitoring systems. The real time (RT) data is often transmitted to remote servers or WITSML (Wellsite Information Transfer Standard Markup Language) ([Energistics 2016](#)) servers, which adhere to established energy standards for data exchange. WITSML provides domain-specific schemas to structure and transfer data, enabling interoperability across different software and tools in the drilling ecosystem.

To derive proper algorithm inferences for automated decision-making, all these diverse data sources need to be aggregated. The aggregation process consolidates data from multiple streams, such as drilling logs, well trajectories, and sensor readings, into centralized workflows. For example, MWD data include formation evaluation metrics like gamma ray and resistivity, while surface sensors capture torque, pressure, and rotation speed. These data streams are pushed to WITSML servers in real-time to facilitate remote monitoring and decision-making.

Traditional BI solutions as shown in [Fig. 1](#) rely heavily on data pipelines and aggregation workflows to process this data into actionable insights. These pipelines often involve multiple processing layers and platforms to integrate data from disparate sources. However, building and maintaining such workflows require deep domain knowledge of the underlying data sources and schemas. This complexity often necessitates collaboration between data engineers and domain experts, as traditional analytics engineers may lack the necessary expertise in drilling-specific data standards such as WITSML.

As a result, these workflows tend to be static and difficult to extend. Any new requirements, such as adding a new data source or creating a custom workflow, require additional resources and time to design, implement, and validate. This not only increases operational costs but also delays the availability of insights. Despite recent advancements, many BI platforms still struggle to provide real-time adaptability and scalability, often relying on pre-defined dashboards and rigid workflows that are unable to meet the dynamic demands of modern drilling operations or the specificities of particular drilling preferences.

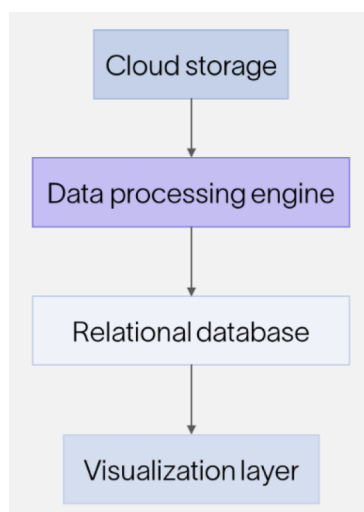


Figure 1—Standard BI solution.

Evolution of RAG systems

Retrieval-Augmented Generation (RAG) augments Large Language Models (LLMs) with real-time data retrieval, addressing the static nature of pre-trained models. Traditional RAG systems, though effective, have linear and static workflows where tasks follow a fixed, sequential process without adaptation to new information during execution. Queries are processed through predefined pipelines, and their outputs remain static, limiting flexibility. They struggle with multi-step reasoning and complex tasks, limiting scalability in dynamic industrial settings. Newer variants, such as Graph RAG, leverage knowledge graphs for structured context, while Agentic RAG introduces agent-based methods-like reflection and planning-to handle intricate workflows. According to the literature (Singh et al. 2025), these advances enable more accurate retrieval, adaptability, and interactive applications.

One key innovation in Agentic RAG is the convergence of agentic workflows with RAG, which enables systems to adaptively select tools, collaborate across agents, and manage diverse data modalities. For example, the framework introduced in paper (Lee et al. 2024) demonstrates how agentic patterns can be applied to solve hybrid questions that involve both textual and relational data. By incorporating self-reflection and validation mechanisms, Agentic RAG systems can improve accuracy and reduce hallucinations, making them highly effective in industrial contexts.

Need for Agentic RAG in Drilling

The dynamic and complex nature of drilling operations necessitates a system capable of integrating diverse data sources, providing real-time insights, and adapting to evolving operational demands. Drilling workflows generate a vast array of structured and unstructured data, including well, wellbore, rig, and operational context data. These are further enriched by domain-specific models, such as Bottom Hole Assembly (BHA) run and tubular models, which describe the tools and configurations used during drilling operations. However, analyzing and correlating these data streams in real time poses significant challenges.

The Agentic RAG framework addresses these challenges by combining LLM-driven reasoning with a graph-based knowledge model. This approach enables the integration of fragmented data streams into a unified schema, allowing for efficient contextualization and multi-step workflows. By capturing both synchronous and asynchronous events, such as telemetry data acquisition, tool usage, and unexpected operational changes, the system ensures that all relevant information is readily accessible to key stakeholders in real time.

For example, automated steering tendency estimation, as discussed in (BA et al. 2024) indicates the critical need for real-time analysis of drilling tendencies to optimize wellbore trajectory. A directional

driller (DD) leveraging the Agentic RAG framework can access real-time steering tendency metrics and correlate them with BHA run data to make trajectory corrections efficiently. The ability to dynamically expose and analyze KPIs such as steering tendencies and drilling trajectory metrics in one system removes the dependency on multi-modal statistical analysis, which is often cumbersome and static.

Furthermore, the integration of performance metrics for drilling operations, as outlined in the paper (Murphy, Petrova and Jane 2024), underscores the role of key performance indicators (KPI) in evaluating operational efficiency. The Agentic RAG system enables continuous monitoring and correlation of multiple KPIs, such as rate of penetration (ROP), tool wear, torque, and pressure. For instance, a remote operations engineer can dynamically assess how a drop in ROP correlates with an increase in torque or tool wear in real time. This real-time contextual analysis allows for proactive interventions, reducing downtime and operational risks.

The Agentic RAG framework leverages a knowledge graph to deliver on-demand KPIs, adapting to dynamic drilling conditions and varied user personas. Unlike static systems, it evolves with operations, reducing multi-modal complexity and manual intervention. By providing real-time, context-aware insights, it boosts decision-making, efficiency, and safety, while minimizing reliance on specialized tools.

Graph and ontology Design

Overview of the Knowledge Graph

The knowledge graph is central to the Agentic RAG framework, providing a unified schema that integrates domain-specific data with event-driven insights. This ontology is designed to address the complexity of drilling operations by combining structured and dynamic data elements. Fig. 2 shows a subset of this ontology.

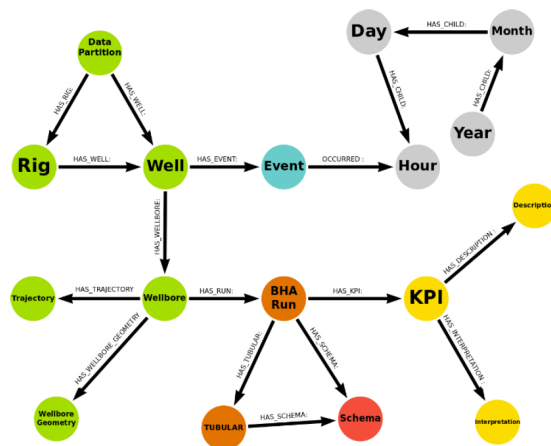


Figure 2—Subset of the graph ontology design.

Key entities and relationships in the ontology include:

- **Well:** Represents individual drilling sites with attributes such as depth, status, geographic location and wellbore geometry. It acts as a hub connecting related operational and event data.
- **Rig:** Captures operational details about the equipment, its configuration, and associated deployments.
- **Service:** Represents deployment details, tool usage, and maintenance records, enabling analysis of operational efficiency.
- **KPI:** Tracks critical metrics like ROP and Downlink Execution Time, offering performance insights into drilling operations.

Key relationships include:

- HAS_EVENT: Connects wells or rigs to events for tracking operational history.
- DEPLOYED_ON: Links rigs to wells to provide deployment context.
- TRACKS: Associates services with their respective KPIs and performance metrics.

Event-Driven Ontology for Drilling Operations

The ontology integrates domain-specific data with event-driven tracking to capture the dynamic nature of drilling operations. For example, directional drilling relies heavily on telemetry data from tools such as RSS (Rotary Steerable System) and MWD (Measurement While Drilling). Events such as Downlink Configuration Changes or Steering Adjustments are logged and linked to specific wells, providing granular insight into operational performance. Each event is enriched with metadata such as timestamps, container IDs (e.g., Wellbore ID), and operational context, enabling advanced querying and analytics.

Design of the Agentic RAG Framework

Multi-Agent Orchestration

The Agentic RAG Framework employs a graph-enhanced multi-agent orchestration design that addresses the challenges of dynamic data processing in drilling operations. By combining specialized agents with a graph-based knowledge model, the system enables context-aware queries, seamless data integration, and actionable insights. This approach builds on recent advances in RAG systems and agentic design principles, as discussed in the paper (Lee et al. 2024) and studies on agent-based RAG systems using graph augmentation (Jeong 2025).

Orchestration Pattern and Design. The architecture as shown in Fig. 3 follows an orchestration pattern where a Supervisor Agent centrally manages interactions among multiple sub-agents. Each agent is implemented using GPT-4o, with specialized role designed to handle distinct tasks in a modular and efficient manner. The design leverages the knowledge graph as a unified representation of entities and relationships in drilling operations, such as rigs, wells, and telemetry events, acting as knowledge base providing agent with additional domain context for information retrieval.

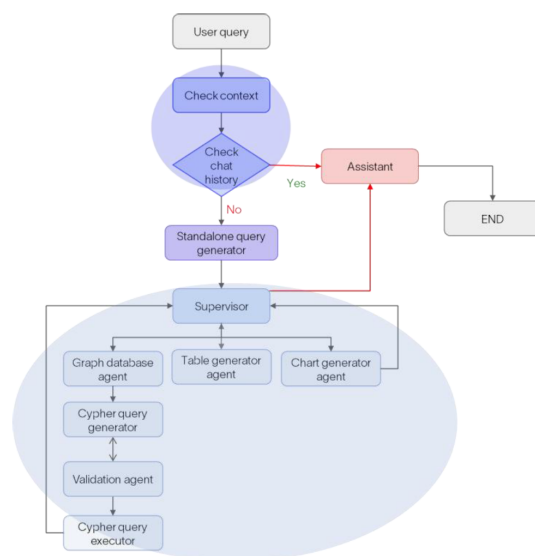


Figure 3—Multi-agent system architecture design.

In addition to the knowledge graph, the system incorporates a memory handling mechanism where chat history is stored as an evolving context knowledge base. As the system interacts with users, the chat history is continuously expanded, forming an additional RAG resource for improved information retrieval over time. Fig. 4 shows an illustration of RAG consisting of chat history and knowledge graph database.

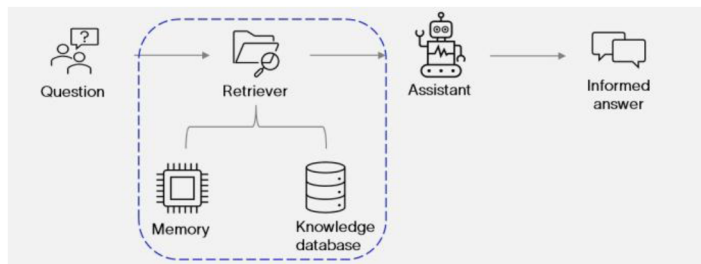


Figure 4—Example of retrieval-augmented generation.

The Assistant functions as the primary interface for user interaction, receiving inputs from both the system memory and the Agentic RAG framework. Upon a user query, the system first checks the chat history for a suitable answer; if found, it is immediately returned via the Assistant. Otherwise, the query is augmented with relevant context from the chat history and provided context, forming a standalone version. This standalone query is then passed to the Supervisor, which orchestrates designated sub-agents for task execution. Each sub-agent processes its assigned task and reports back to the Supervisor, which aggregates the results and forwards them to the Assistant for final user delivery.

Roles of Sub-Agents and Interactions. The Agentic RAG framework operates through a cohesive interplay of specialized sub-agents, each fulfilling a dedicated role within the system. By leveraging the modular nature of these agents, the framework ensures that complex user queries are handled efficiently.

Supervisor Agents

The Supervisor Agent acts as the central orchestrator of the system. Upon receiving a user query, it determines which sub-agents are required to fulfill the request and coordinates their tasks. For instance, when a user requests, "Generate a performance report for Rig A over the last quarter," the Supervisor Agent delegates the query to the Graph Database Agent for retrieving event data and the Chart Generator Agent for creating a corresponding visualization. The Supervisor ensures that each sub-agent's output is aggregated into a unified response.

Graph Database Agents

The Graph Database Agent interfaces directly with a native graph database in our case [Neo4j](#) (Neo4j), translating user queries into Cypher commands to retrieve relevant data. Cypher, a declarative query language designed specifically for graph databases, excels at representing and querying relationships between entities. Its intuitive syntax allows the agent to focus on extracting meaningful insights without relying on complex nested queries or joins, as seen in traditional SQL (Structured Query Language) systems. Specifically, based on the user's query and the provided context including schema and event descriptions, this agent generates Cypher queries which are then executed to retrieve the relevant data from the graph database, ensuring accurate and contextually relevant responses to user queries.

This design ensures that complex relationships-such as those involving time, events, and rigs-are efficiently retrieved from the graph model, providing precise and actionable insights for drilling operations. For example, a graph database agent can retrieve rig information per quarter for the end user:

- User Query: How many rigs are used in the first and the second quarter respectively in 2024?
- Cypher Query from LLM:


```

MATCH (r:Rig)-[:HAS_WELL]->(w:Well)
-[:HAS_EVENT]->(e:Event)-[:OCCURRED]
->(h:Hour)-[:HAS_CHILD]->(d:Day)
<-[:HAS_CHILD]->(m:Month)-[:HAS_CHILD]
-(y:Year)
WHERE y.number = 2024
AND m.number IN [1, 2, 3]
RETURN 'Q1' AS Quarter,
count(e) AS numberOfRigs
UNION
MATCH (r:Rig)-[:HAS_WELL]->(w:Well)
-[:HAS_EVENT]->(e:Event)-[:OCCURRED]
->(h:Hour)-[:HAS_CHILD]->(d:Day)
<-[:HAS_CHILD]->(m:Month)-[:HAS_CHILD]
-(y:Year)
WHERE y.number = 2024
AND m.number IN [4, 5, 6]
RETURN 'Q2' AS Quarter,
count(e) AS numberOfRigs

```

- Answer: Graph search finished. In 2024, the number of rigs used in the first and the second quarter respectively is as follows:
 - Q1:219,902 rigs.
 - Q2:50,415 rigs.

Furthermore, by harnessing the advanced natural language understanding capabilities of LLMs, user-proposed general queries can be effectively interpreted and transformed into well-structured Cypher queries. This facilitates the generation of informative and relevant results, enhancing the overall user experience.

- User Query: I want to know the general statistics of event generated for the well "WellId"?
- Cypher Query from LLM:

```

MATCH (w:Well {Id: 'WellId'})
-[:HAS_EVENT]->(e:Event)
RETURN e.value AS eventName,
count(e) AS eventCount
LIMIT 1000

```

- Answer: Graph search finished. The general statistics of events generated for the well 'WellId' are as follows:
 - There are 64 ssevents with no specific name.

Chart Generator Agents

The Chart Generator Agent serves as a crucial component in delivering visual clarity to drilling operations, dynamically creating visual representations such as bar charts, pie charts, heat maps, and trend lines as demonstrated in the paper (Ye et al. 2024). The agent can generate JavaScript code for real-time rendering by front-end frameworks directly in the browser. This approach leverages the power of LLMs to

dynamically craft code as presented by (Yu et al. 2024), making it particularly well-suited for integrating with visualization libraries like Plotly (Plotly Technologies Inc 2013) or D3.js (D3.js 2011).

In Figure 5, the Chart Generator Agent generates JavaScript code compatible with Plotly.js. The front-end framework then utilizes this generated JavaScript code to render the plot on the interface. To ensure the security and correctness of the plot, the front-end framework has been extended to support additional steps for validating and executing JavaScript code.

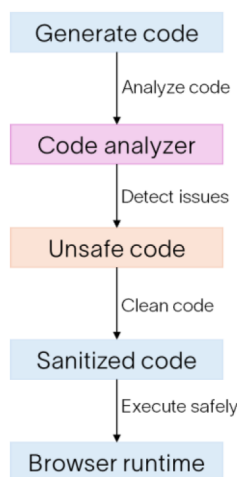


Figure 5—Code execution for chart generation.

For instance, after the Graph Database Agent retrieves event data for a rig, the Chart Generator Agent translates this data into interactive visualizations. A pie chart could be rendered to illustrate downtime causes across multiple rigs, or a trend line might display ROP performance metrics over a selected time range. These visualizations are not static; users can interact with them in real-time, enabling deeper insights and exploration of the data.

Validation Agents

The Validation Agent is central to maintaining accuracy, reliability, and performance within the Agentic RAG Framework. Its primary role is to verify that data retrieved by the Graph Database Agent meets predefined standards and that Cypher queries execute efficiently. By analyzing Neo4j's query execution plans, the Validation Agent detects performance bottlenecks—such as overly large data retrieval or inefficient patterns—before execution occurs, similar techniques are shown in the paper (Zheng et al. 2024). For example, if a query attempts to scan millions of events in a single operation, the agent refines the query to limit the result set or break it into smaller segments. Similarly, if a user's query is too ambiguous, it provides feedback to refine the input.

Cypher Query Validation Example

1. User Query: "Retrieve all downtime events for Rig B in the past six months."
2. Validation Agent: Pre-evaluates the Cypher query.
3. Action: If necessary, the agent works with the Cypher Query Generator to optimize the query or send feedback to the user to refine their request.
4. Result: The optimized query is executed, providing accurate and actionable insights without performance issues.

o Initial Query:

```

MATCH (r:Rig {name: 'Rig B'})
-[:HAS_EVENT]->(e:Event)
  
```



```

-[:OCCURRED]->(t:Time)
WHERE e.eventType = 'Downtime'
AND t.timestamp >= date('2024-07-01')
AND t.timestamp <= date('2024-12-31')
RETURN e.name AS EventName,
e.timestamp AS Timestamp,
e.details AS Details
ORDER BY e.timestamp ASC

```

- Corrected Query with Limit:

```

MATCH (r:Rig {name: 'Rig B'})-[:HAS_EVENT]
->(e:Event)-[:OCCURRED]->(t:Time)
WHERE e.eventType = 'Downtime'
AND t.timestamp >= date('2024-07-01')
AND t.timestamp <= date('2024-12-31')
RETURN e.name AS EventName,
e.timestamp AS Timestamp,
e.details AS Details
ORDER BY e.timestamp DESC
LIMIT 100

```

By incorporating Neo4j's execution planning tools and dynamic query adaptation ([Neo4j](#)), the Validation Agent ensures that both data accuracy and system performance are upheld.

Results and Evaluation

Performance Metrics

The framework was tested using a dataset of 3 million tracking events and 1,000 wells. The evaluation of the test workflow was conducted on an Azure Virtual Machine (VM) deployed in the East US region, alongside a Neo4j graph database deployed in the same region. The LLMs utilized in the workflow are GPT-4o provided by Azure OpenAI.

The framework evaluates key performance metrics to measure the effectiveness of the Agentic RAG system. Query Accuracy tracks the system's ability to generate the correct Cypher query based on user input, ensuring the right data is retrieved. Query Latency measures the time taken from user input to the final response, covering query generation and execution. Additionally, Chart Generation Accuracy evaluates the chart agent's capability to generate correct JavaScript code for rendering charts and visualizations on the front-end.

Query Accuracy. The system achieved an overall accuracy of 97.14% across a benchmark set of 70 test queries, as detailed in [Table 1](#). The queries were categorized into three complexity levels: Basic, Medium, and Complex (Hard), following a query complexity evaluation approach similar to that presented in ([Kotiranta, Junkkari and Nummenmaa 2022](#)). The Basic level included 15 queries involving simple Cypher query generation targeting a single node type, such as listing all well id values, achieving 100% accuracy. The Medium level, consisting of 25 queries, required generating Cypher queries that established connections between multiple entities and incorporated time-based conditions, such as correlating well operations with specific events. The Complex (Hard) level included 30 queries requiring advanced query generation, such as interpreting timestamps, handling multiple relationships, and mapping across four or more entity types, with no upper limit.

Table 1—Cypher query generation accuracy result.

Accuracy / Complexity	Basic	Medium	Hard
Accuracy before validation	100.00%	92.00%	90.00%
Accuracy after validation	100.00%	96.00%	93.33%

- Basic Level Test Query

- User Query: List all wells by their id and name.
- Cypher Query from LLM:

```
MATCH (w:Well) RETURN w.ID AS WellId,
w.name AS WellName
```

- Medium Level Test Query

- User Query: For a given well "WellId". list all events along with their Hour nodes to see at what time they occurred.
- Cypher Query from LLM:

```
MATCH (w:Well {id:'WellId'})-
[:HAS_EVENT]->(e:Event)-[:OCCURRED]
->(h:Hour)
RETURN e,h
```

- Hard Level Test Query

- User Query: Find all "data aggregation" scoped events that happened on hours where "steering", "recommendation" also occurred. Include all Well-related nodes (Rig, Rig Deployment, Town Deployment, Services, Versions) and the complete time chain.
- Cypher Query from LLM:

```
MATCH (e1:Event {scope:
data aggregation})-[:OCCURED]->
(h:Hour)<-[:HAS_CHILD]-(d:Day)<-
[:HAS_CHILD]-(m:Month)<-
[:HAS_CHILD]-(y:Year)
MATCH (e2:Event {scope: 'steering',
event_type: recommendation})-
[:OCCURED]->(h)
MATCH (w:Well)-[:HAS_EVENT]->(e1)
OPTIONAL MATCH (w)-
[:HAS_RIGDEPLOYMENT]->
(rd:RigDeployment)
OPTIONAL MATCH (w)-
[:HAS_TOWNDEPLOYMENT]->
(td:TownDeployment)
OPTIONAL MATCH (w)-
[:HAS_SERVICEDEPLOYMENT]->
```

```
(de:DeploymentEvent)-
[:HAS_SERVICE]->(s:Service)
RETURN e1, e2, w, rd, td, s, de, h,
d, m, y
```

The validation tests revealed that the LLM exhibited robust performance in interpreting user queries and generating valid, executable Cypher queries. The generated Cypher queries demonstrated accurate structure and parameterization, ensuring successful execution. However, further analysis identified that issues originated from poorly formatted user inputs, which led to ambiguities in the LLM's interpretation and subsequent query generation. The validation agent is able to capture and report the issue back to the user and regenerate the query correctly. This highlights the critical impact of human input quality on the performance of the LLM in task execution. Additionally, queries containing examples need to be clearly delineated and explicitly stated to ensure that the LLM can effectively distinguish between illustrative examples and the designated parameters to be retrieved.

Chart Generation Accuracy. The system was evaluated using 50 queries designed for testing a diverse range of chart-types generation, including bar charts, pie charts, line plots, box plots, and heatmaps, as well as tabular representations etc. Fig. 6 illustrates an example of a pie chart generated by the system in response to a user query: "Calculate the percentage of tool telemetry remaining InSync versus NoSignal or Syncing."

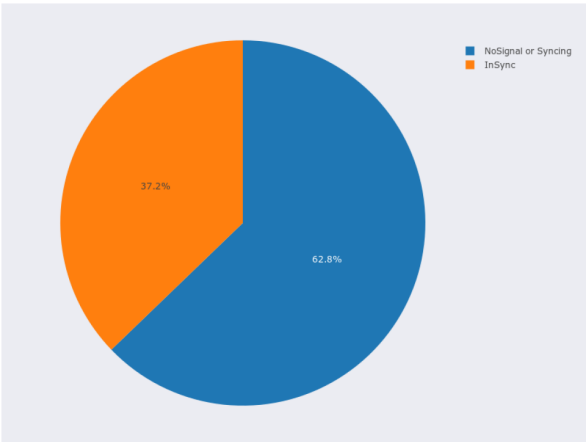


Figure 6—Example of pie chart generated by Agentic RAG system.

Query Latency. In this section, three query latency metrics are presented. The end-to-end query latency measures the total duration taken to process a user query, from the initial input through the generation of the Cypher query, its execution, and the return of the final result. Within this process, Cypher query generation latency tracks the time required for the LLM to create the appropriate query based on user input, while Cypher query execution latency measures the time needed to run the query and retrieve the relevant data. As query complexity increases, both the generation and execution phases contribute to variations in the overall response time, reflecting the system's ability to efficiently handle diverse queries. The results are detailed in Table 2.

Table 2—Cypher query latency breakdown result.

Response Time / Complexity	Basic	Medium	Hard
Avg End-to-End (s)	3.06	5.30	7.67
Cypher Generation (s)	1.02	1.69	3.33
Query Execution (s)	0.20	0.54	2.21

Real-World Application

The Agentic RAG Framework has proven transformative across various facets of drilling operations, particularly in enabling more effective decision-making and operational efficiencies in environments where traditional tools fall short. By leveraging the underlying knowledge graph and multi-agent orchestration, the framework empowers different stakeholders with targeted insights tailored to their needs, whether they are support engineers, domain experts, or operations managers.

One of the most impactful applications has been supporting engineering for rigs operating in remote environments where software is deployed on edge devices rather than centralized cloud systems. In such scenarios, accessing and consolidating telemetry data, deployment logs, and operational metadata has historically been a time-intensive and error-prone process. With the framework, support engineers can now query the knowledge graph directly to retrieve a comprehensive overview of deployment details. For example, by querying the status of all software versions deployed across rigs in a particular region, engineers can quickly identify discrepancies, such as outdated deployments or missing updates. This capability has significantly streamlined support workflows, enabling faster resolution of issues and reducing downtime associated with troubleshooting.

The framework has demonstrated significant potential for enhancing the efficiency of complex statistical analyses and real-time monitoring in drilling operations. By integrating telemetry logs with operational data through a knowledge graph, domain experts can execute queries that were previously cumbersome or required manual intervention. For instance, analyzing downlink commands alongside the Rate of Penetration (ROP) performance provides insights into potential optimizations, such as tool configuration adjustments to improve speed or address inefficiencies in communication. Although large-scale case studies are still underway, preliminary applications suggest promising improvements in data-driven decision-making and operational efficiency.

Conclusion and Future Work

The Agentic RAG Framework has the potential of a significant advancement in the digital transformation of drilling operations. Its ability to bridge the gap between disparate data sources through a unified knowledge graph is a cornerstone of its effectiveness, providing a comprehensive view of the data ecosystem. This capability is empowering stakeholders across the drilling lifecycle, from support engineers troubleshooting deployments in remote rigs to domain experts conducting sophisticated analyses of ROP performance and tool configurations. The result improves reporting accuracy, enhanced decision-making, and optimized production strategies, fundamentally transforming the management of drilling operations.

By orchestrating multiple intelligent agents, the framework has demonstrated its ability to tackle diverse challenges, such as telemetry monitoring, predictive maintenance, and workflow optimization. The seamless integration of knowledge graphs with natural language interfaces has not only streamlined workflows but also enabled stakeholders to derive actionable insights in real time. These achievements underscore the framework's potential to address the complexities of modern drilling environments, where dynamic and data-intensive workflows demand adaptive and intelligent solutions.

Looking ahead, the framework's adaptability paves the way for specialized agents capable of tackling increasingly complex, domain-specific tasks. For instance, agents that interpret geological data or simulate drilling scenarios can advance beyond mere analysis, actively supporting directional drillers in decision-making. By leveraging the knowledge graph, the framework can evolve to include prescriptive analytics—delivering real-time optimizations and actionable recommendations, such as trajectory adjustments or tool-usage strategies. Ultimately, incorporating autonomous, self-learning agents, potentially using reinforcement learning and self-criticism, will keep the system effective in rapidly changing environments. This trajectory affirms the Agentic RAG Framework's potential to transform drilling operations and drive sustained innovation in the energy sector.

Reference

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktäschel, T., et al 2020. Retrieval-augmented generation for knowledge intensive NLP tasks. In Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20). Article 793, 9459–9474. Red Hook, NY, USA. <https://doi.org/10.48550/arXiv.2005.11401>
- Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., and Gao, J. Large language models: A survey. 2024. <https://doi.org/10.48550/arXiv.2402.06196>
- Singh, A. 2023. Exploring language models: A comprehensive survey and analysis. 2023 International Conference on Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE), Chennai, India. pp. 1–4. doi: [10.1109/RMKMATE59243.2023.10369423](https://doi.org/10.1109/RMKMATE59243.2023.10369423)
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y., and Wen, J.-R. 2024. A survey of large language models. <https://doi.org/10.48550/arXiv.2303.18223>
- Energistics. 2016. WITSML Data Standards. <https://energistics.org/witsml-data-standards>.
- Singh, A., Ehtesham, A., Kumar, S., and Talaei Khoei, T. 2025. Agentic retrieval-augmented generation: A survey on agentic RAG. Cleveland State University, Department of Computer Science, Ohio, USA. <https://doi.org/10.48550/arXiv.2501.09136>
- Lee, M.-C., Zhu, Q., Mavromatis, C., Han, Z., Adeshina, S., Ioannidis, V. N., Rangwala, H., and Faloutsos, C. 2024. Agent-G: An agentic framework for graph retrieval augmented generation [Withdrawn submission]. ICLR 2025 Conference.
- BA, S., Omr, M., Mansour, D., Sahli, H., and Goel, P. 2024. Automated steering tendency estimation for steering ellipse derivation. Presented at the Mediterranean Offshore Conference, Alexandria, Egypt, October. <https://doi.org/10.2118/223289-MS>
- Murphy, N., Petrova, E., & Jane, K. 2024. Performance metrics for drilling operations: Key performance indicators (KPIs) in drilling. *Tunneling and Trenchless Technology*.
- Jeong, C. 2025. A study on the implementation method of an agent-based advanced RAG system using graph. 2024 *Knowledge Management Research*. <https://doi.org/10.48550/arXiv.2407.19994>
- Neo4j. The graph database platform, <https://neo4j.com/>.
- Ye, Y., Hao, J., Hou, Y., Wang, Z., Xiao, S., Luo, Y., and Zeng, W. 2024. Generative AI for visualization: State of the art and future directions. *Visual Informatics* 8(2): 43–66. <https://doi.org/10.48550/arXiv.2404.18144>.
- Yu, X., Liu, L., Hu, X., Keung, J. W., Liu, J., and Xia, X. 2024. Where are large language models for code generation on GitHub? [10.48550/arXiv.2406.19544](https://doi.org/10.48550/arXiv.2406.19544). <https://doi.org/10.48550/arXiv.2406.19544>.
- Plotly Technologies Inc. 2013. *Data visualization software tools*, <https://plotly.com/>.
- D3.js. 2011. A JavaScript library for producing dynamic, interactive data visualizations in web browsers. <https://d3js.org/>
- Zheng, Y., Dou, W., Tang, L., Cui, Z., Gao, Y., Song, J., Xu, L., Zhu, J., Wang, W., Wei, J., Zhong, H., and Huang, T. 2024. Testing Gremlin-Based Graph Database Systems via Query Disassembling. In Proceedings of the 33rd ACM SIG SOFT International Symposium on Software Testing and Analysis (ISSTA 2024), pages 1695–1707. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3650212.3680392>.
- Neo4j. Understanding execution plans. Neo4j. <https://neo4j.com/docs/cypher-manual/current/planning-and-tuning/execution-plans/> (accessed 17 October 2024).
- Kotiranta, P., Junkkari, M., and Nummenmaa, J. 2022. Performance of graph and relational databases in complex queries. *Applied Sciences*. 12(13). 6490. <https://doi.org/10.3390/app12136490>